Serverless Overviews - AWS SAA-C03 Study Notes

1. Serverless Introduction

- **Definition:** A model where the cloud provider manages the infrastructure (servers, OS, patching, scaling). You focus solely on code.
- **Key Traits:**
  - No infrastructure provisioning or management.
  - Automatic scaling (scale-to-zero).
  - Pay-for-value (pay only when code runs or requests are made).
  - Highly available by default.

2. AWS Lambda Overview

- **Concept:** Function-as-a-Service (FaaS). Run code without provisioning servers.
- **Triggers (Event-Driven):**
  - **Synchronous:** API Gateway, ALB, CLI, SDK (Client waits for response).
  - **Asynchronous:** S3, SNS, EventBridge (Client gets 202 accepted; Lambda handles retries/DLQ).
  - **Poll-Based (Event Source Mapping):** Kinesis, DynamoDB Streams, SQS (Lambda polls the service).
- **Pricing:** Based on **# of requests** and **duration** (GB-seconds: duration * memory allocated).

3. Lambda Limits (Exam Critical)

- **Execution Timeout: 15 minutes** (900 seconds) hard limit.
  - *Architectural fix:* If a task takes >15 mins, use **AWS Batch** or **Step Functions** or EC2.
- **Memory:** 128 MB to **10 GB**. (CPU scales effectively with memory).
- **Deployment Package:**
  - Zipped: **50 MB**.
  - Unzipped: **250 MB**.
  - Container Image: **10 GB**.
- **Payload Size:**
  - Synchronous (e.g., API Gateway): **6 MB**.
  - Asynchronous: **256 KB**.
- **Concurrency:** Default **1,000** concurrent executions per region (soft limit, can be raised).

4. Lambda Concurrency

- **Reserved Concurrency:**
  - Guarantees a specific number of concurrent instances for a function.
  - **Acts as a limit:** The function cannot exceed this cap (good for protecting downstream DBs).
- **Provisioned Concurrency:**
  - Pre-initializes execution environments.
  - **Solves "Cold Start"** latency issues.
  - Use case: High-frequency trading apps, interactive APIs needing strict low latency.

5. Lambda SnapStart

- **Goal:** Dramatically reduce cold start latency (up to 90%) for **Java**, **Python**, and **.NET** functions.
- **Mechanism:** It creates a snapshot of the initialized microVM (memory/disk state) and caches it.
- **Cost:** No extra cost for the feature itself (standard duration/storage apply).
- **Constraint:** Does NOT support Node.js or Ruby (as of current exam scope).

6. Lambda@Edge vs. CloudFront Functions

- **Use Case:** Running logic at the edge (closer to the user) to customize content delivery.

Feature	CloudFront Functions	Lambda@Edge
Runtime	JavaScript (ES5) only	Node.js, Python
Execution Time	Sub-millisecond (< 1ms)	Up to 5 sec (Viewer) / 30 sec (Origin)
Scale	Millions of requests/sec	Thousands of requests/sec
Network Access	**No** (Cannot call external APIs)	**Yes** (Can call DynamoDB, S3, APIs)
Use Cases	Header manipulation, URL rewrites, JWT validation (lightweight), Cache key normalization.	Complex logic, resizing images, accessing databases, advanced authentication, calling 3rd party APIs.

7. Lambda in VPC

- **Default Behavior:** Lambda runs outside your VPC (has internet access but cannot access private VPC resources like RDS).
- **VPC Configuration:** To access private resources (RDS, ElastiCache), you must attach the Lambda to a VPC.
  - Requires: **Private Subnets** and **Security Groups**.
- **Internet Access Requirement:**

- A Lambda in a VPC **loses default internet access**.
- To regain internet access (e.g., to call DynamoDB public endpoints or 3rd party APIs), the VPC must have a **NAT Gateway** in a public subnet.
- *Exam Trap:* Do NOT put Lambda in a public subnet to give it public IP access; it won't work. It must go Private Subnet -> NAT Gateway.

8. RDS & Lambda Integration

- **Invoking Lambda from RDS:**
  - PostgreSQL and Aurora MySQL can invoke Lambda functions directly from SQL queries (stored procedures).
  - Use case: Data processing during ingestion, sending emails when a row is inserted.
- **RDS Event Notifications:**
  - RDS sends notifications about the *database instance itself* (e.g., "DB shutdown", "Failover started") to **SNS**.
  - SNS can then trigger Lambda.
  - *Exam Tip:* This is for operational events, NOT data changes. For data changes, you usually polling or app-level logic, or DynamoDB.

9. Amazon DynamoDB (Serverless NoSQL)

- **Basics:** Key-Value and Document DB. Millisecond latency at any scale.
- **Primary Key:**
  - **Partition Key (HASH):** Determines data distribution.
  - **Sort Key (RANGE):** Sorts data within the partition.
- **Read Consistency:**
  - **Eventual Consistency (Default):** Fast, half the cost. Data might be stale (<1s).
  - **Strongly Consistent:** Guarantees latest data. Higher latency, double the read cost (RCUs).
- **Capacity Modes:**
  - **Provisioned:** You specify WCU/RCU. Use Auto Scaling. Good for predictable loads.
  - **On-Demand:** Scales automatically. Good for unpredictable/spiky loads. More expensive per request.

10. DynamoDB Advanced Features

- **DAX (DynamoDB Accelerator):**
  - Fully managed **in-memory cache** for DynamoDB.
  - Improves performance from milliseconds to **microseconds**.
  - Transparent (no code changes needed).
- **DynamoDB Streams:**

    - Time-ordered sequence of item-level modifications (INSERT, UPDATE, DELETE).
    - Retention: **24 hours**.
    - Primary Target: **Lambda** (to process changes, replicate data to OpenSearch, etc.).
- **Kinesis Data Streams (for DynamoDB):**
    - Alternative to DynamoDB Streams for capturing item-level changes.
    - Retention: Up to **1 year** (vs 24h for DynamoDB Streams).
    - Feature: Supports higher throughput and multiple consumers (fan-out) without throttling.
- **Global Tables:**
    - **Multi-Region Active-Active** replication.
    - Writes in any region are replicated to all other regions (~1 sec).
    - Requires DynamoDB Streams to be enabled.
- **TTL (Time To Live):**
    - Automatically deletes items after a specific timestamp (stored in an attribute) expires.
    - Free of charge (doesn't consume WCUs). Great for session storage.

11. Amazon API Gateway

- **Role:** The "front door" for serverless applications. Exposes Lambda, HTTP endpoints, or AWS services.
- **Endpoint Types:**
    - **Edge-Optimized:** Default. Uses CloudFront edge locations (global clients).
    - **Regional:** Clients in the same region.
    - **Private:** Accessible only from within a VPC (via Interface VPC Endpoint).
- **Integrations:**
    - **Lambda Proxy:** Passes the raw request to Lambda. Lambda returns the raw response. (Preferred for flexibility).
    - **Mock:** Returns a response without sending to backend (good for testing).
- **Features:**
    - **Caching:** Caches responses to reduce load on backend (TTL default 300s).
    - **Throttling:** Protects backend limits (Token Bucket algorithm).
    - **Stages:** (e.g., Dev, Prod, v1, v2). Can use "Canary Release" to send % of traffic to new stage.

12. AWS Step Functions

- **Concept:** Serverless orchestration service. Models workflows as **State Machines**.
- **Language:** Defined using **Amazon States Language (ASL)** (JSON-based).
- **Use Cases:** Long-running processes, human approval steps, chaining Lambdas, error handling (Retry/Catch).

- **Standard vs. Express Workflows:**
  - **Standard:** Long-running (up to **1 year**), Exactly-once execution. Good for order fulfillment.
  - **Express:** Short-lived (up to **5 mins**), At-least-once execution. Good for high-volume event processing (IoT ingestion).

13. Amazon Cognito Overview

- **User Pools (Authentication - "Who are you?"):**
  - User directory (Sign up, Sign in).
  - Supports MFA, Password policies, Social Sign-in (Google, Facebook, SAML).
  - Returns a **JWT (JSON Web Token)**.
- **Identity Pools (Authorization - "What can you do?"):**
  - Exchanges a login token (from User Pools or Social) for **Temporary AWS Credentials**.
  - Maps users to **IAM Roles**.
  - Allows users to access AWS services directly (e.g., Upload file to S3 bucket).
- **Web Identity Federation Pattern:**
  1. User logs in to Cognito User Pool (or Google).
  2. App receives Token.
  3. App sends Token to Cognito Identity Pool.
  4. Identity Pool returns AWS Credentials (Access Key/Secret Key).
  5. App accesses S3/DynamoDB using those credentials.